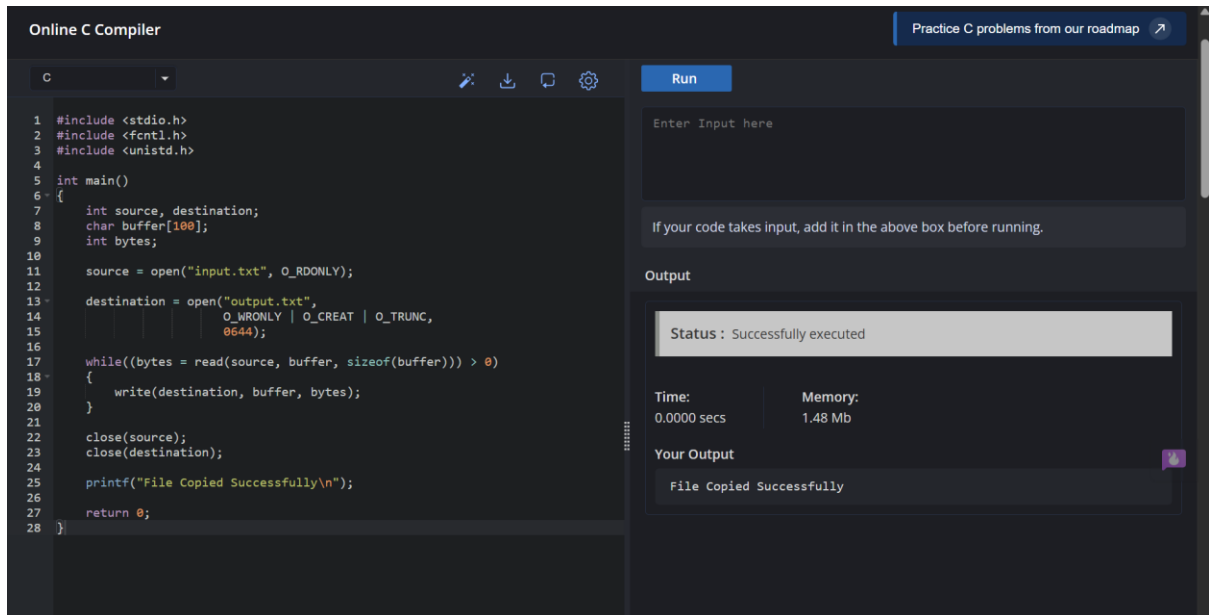


EXPERIMENT 02:

2. Identify the system calls to copy the content of one file to another and illustrate the same using a C program.



The screenshot displays an online C compiler interface. On the left, a code editor shows a C program that opens 'input.txt' in read mode and 'output.txt' in write mode, then copies the content of 'input.txt' to 'output.txt' using a buffer and a while loop. The program prints 'File Copied Successfully' and returns 0. On the right, the 'Run' button is visible. Below it, the 'Output' section shows the status 'Successfully executed', execution time '0.0000 secs', memory usage '1.48 Mb', and the output 'File Copied Successfully'.

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4
5 int main()
6 {
7     int source, destination;
8     char buffer[100];
9     int bytes;
10
11     source = open("input.txt", O_RDONLY);
12
13     destination = open("output.txt",
14                       O_WRONLY | O_CREAT | O_TRUNC,
15                       0644);
16
17     while((bytes = read(source, buffer, sizeof(buffer))) > 0)
18     {
19         write(destination, buffer, bytes);
20     }
21
22     close(source);
23     close(destination);
24
25     printf("File Copied Successfully\n");
26
27     return 0;
28 }
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.48 Mb

Your Output

File Copied Successfully

PSEUDO CODE :

Start

Open source file in Read mode

Open destination file in Write mode

While source file is not empty

 Read data

 Write data into destination file

Close both files

Display "File Copied Successfully"

Stop

Source File : input.txt

Destination : output.txt

File Copied Successfully.

CODE:

```
#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>


int main()
{
    int source, destination;

    char buffer[100];

    int bytes;


    source = open("input.txt", O_RDONLY);


    destination = open("output.txt",
        O_WRONLY | O_CREAT | O_TRUNC,
        0644);


    while((bytes = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytes);
    }


    close(source);

    close(destination);
```

```
printf("File Copied Successfully\n");
```

```
return 0;
```

```
}
```

```
}
```

The screenshot shows the CodeChef C online compiler interface. The code editor on the left contains a C program that uses the `fork()` system call to create a child process. The program prints the PID and PPID for both the parent and child processes. The output panel on the right shows the execution results, including the status, time, memory usage, and the printed output.

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main()
5 {
6     pid_t pid;
7
8     pid = fork();
9
10    if(pid == 0)
11    {
12        printf("\nChild Process");
13        printf("\nPID : %d", getpid());
14        printf("\nPPID : %d\n", getppid());
15    }
16    else if(pid > 0)
17    {
18        printf("\nParent Process");
19        printf("\nPID : %d", getpid());
20        printf("\nPPID : %d\n", getppid());
21    }
22    else
23    {
24        printf("Fork Failed");
25    }
26
27    return 0;
28 }
```

Run

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.768 Mb

Your Output

Parent Process
PID : 1601
PPID : 1600

Child Process
PID : 1602
PPID : 1